

MySQL

MySQL Tutorial for Beginners

PRAVEEN NAIR

Introduction to MySQL 8.1

MySQL is a relational database that stores data like products, categories and customers using tables. A table contains columns and rows. It is like a spreadsheet.

A table may relate to another table using a relationship. For example: one-to-one and one-to-many relationships.

MySQL was created by a Swedish company, MySQL AB and first release was on 23 May 1995

In January 2008, Sun Microsystems bought MySQL AB for \$1 billion.

In April 2009, Oracle Corporation entered into an agreement to purchase Sun Microsystems.

Install MySQL

1. Visit <https://www.mysql.com/downloads/>
2. Download MySQL Installer
3. Connect to MySQL Using mysql command-line client
4. Download sample database classicmodels
5. `mysql> source c:\download\mysqlsampledatabase.sql`
6. `show databases`
7. `use classicmodels`

Querying and Sorting Data

1. show databases;
2. System cls;
3. use classicmodels;
4. show tables;
5. desc customers;
6. select * from customers;
7. select CustomerName, City, Country from Customers;
8. select distinct country from customers;
9. select count(country) from customers;
10. select count(distinct country) from customers;
11. select customername,creditlimit from customers order by creditlimit desc
12. select customername,country,creditlimit from customers order by
country,creditlimit;

Filtering Data

1. = > < >= <= <> BETWEEN LIKE IN
2. select * from customers where country = 'USA'
3. select CustomerName, City, Country from customers
4. where country != 'USA'
5. select CustomerName, City, Country from customers where country in ('USA', 'UK')
6. select CustomerName, City, Country from customers where country not in ('USA', 'UK')
7. select * from payments where amount > 1000
8. select CustomerName, city from customers where CustomerName like '%Ltd%';
9. select customername, country, creditlimit from customers where country like '____tr%';
10. select customername, city, state, country from customers limit 10 offset 2;
11. select customername, city, state, country from customers limit 2, 10;
12. select customername, creditlimit from customers order by creditlimit desc limit 3;
13. select CustomerName, City, Country from customers where country = 'USA' or country = 'UK';
14. select CustomerName, City, Country, creditlimit from customers where country = 'USA' and creditlimit > 100000;
15. select CustomerName, City, Country, creditlimit from customers where not country = 'USA';
16. select customername, addressline1, addressline2 from customers where addressline2 is null;

Creating Database and Tables

```
create database besant
use besant

create table students(id int,name varchar(100),
email varchar(100));

show tables;

desc students;

select * from students;

insert into students(id,name,email)
values(1,Vinod,Vinod@gmail.com)

select * from students;

set sql_safe_updates = 0

update students set email='vinod@hotmail.com'
where id=1

select * from students;

insert into students(id,name,email)
values(2,Sebastian,Sebastian@gmail.com)

select * from students;

delete from students where id=2

select * from students;

create table result(id int,score int)

select * from result

insert into result(id,score) values(1,90)

select * from result

select s.id,name,email,score
from students s inner join result r
on s.id = r.id

select s.id,name,email,score
from students s left join result r
on s.id = r.id

select s.id,name,email,score
from result r right join students s
on s.id = r.id

alter table students
add city varchar(100)

select * from students

update students set city='Chennai'
where id=1
```

```
from result students(s.id,name,email,city)
select s, r from result r join students s
select * from students
```

SQL (Structured Query Language)

1. Define the database and its objects such as tables, views, triggers and stored procedures.
2. Contains statements that updates and query data.
3. Grant a user the permissions to access specific data in the database.

Comments

-- select * from customers

/*select customername
From custoemrs */

Grouping Data

```
SELECT Country,COUNT(CustomerName)
FROM Customers
GROUP BY Country
having COUNT(CustomerName) > 10
order by COUNT(CustomerName);
```

```
.....
select country,city,sum(creditlimit)
from customers
group by country,city
with rollup;
```

Case Statement

```
select customername,creditlimit,
```

```
case
```

```
  when creditlimit > 100000 then 'Good Credit'
```

```
  when creditlimit between 100000 and 90000 then 'Average credit'
```

```
  else 'Bad credit'
```

```
end as creditscore
```

```
from customers;
```

MySQL Functions

`select min(quantityinstock) from products;`

`select max(quantityinstock) from products;`

`select avg(quantityinstock) from products;`

`select count(quantityinstock) from products;`

`select sum(quantityinstock) from products;`

Null Functions

```
SELECT customername,ifnull(addressline2,'NA')
```

```
from customers
```

```
where addressline2 is null
```

```
SELECT customername,4+ifnull(addressline2,0)
```

```
from customers
```

```
where addressline2 is null
```

```
SELECT customername,COALESCE(addressline2,'NA')
```

```
from customers
```

```
where addressline2 is null
```

```
SELECT customername,4+COALESCE(addressline2,0)
```

```
from customers
```

```
where addressline2 is null
```

String Functions

--CHARACTER_LENGTH

```
select char_length(country)
```

```
from customers where country='USA';
```

```
SELECT CONCAT(Addressline1, ' ', city, ' ', country) AS Address
```

```
FROM Customers;
```

```
SELECT CONCAT_WS('-', Addressline1, city, country) AS Address
```

```
FROM Customers;
```

```
select format(creditlimit,2) from customers;
```

```
SELECT INSERT(customername,1,0,'hello') from customers where customername like '%kelly%';
```

lcase,lower,ucase,upper - for capital and small

```
SELECT left(customername,3) from customers where customername like '%kelly%'; --try right
```

```
SELECT length(customername) from customers where customername like '%kelly%';
```

Math Functions

`select abs(-10)`

`select rand()`

`select floor(avg(creditlimit)) from customers`

`select ceil(avg(creditlimit)) from customers`

`SELECT GREATEST(3, 12, 34, 8, 25);`

`SELECT least(3, 12, 34, 8, 25);`

`select mod(5,2) -- remainder`

`select power(4,2) -- pow also works`

`select round(4.9)`

`select sqrt(9)`

`SELECT TRUNCATE(875.378, 2);`

Date Functions

```
select CURDATE(),customername
```

```
from customers;
```

```
curtime(),day(orderdate),month,year,hour,minute
```

```
select datediff(curdate(),orderdate)
```

```
from orders;
```

Subqueries

SELECT

lastName, firstName, email, jobtitle

FROM

employees

WHERE

officeCode IN (SELECT

officeCode

FROM

offices

WHERE

country = 'UK');

SELECT

MAX(items),

MIN(items),

FLOOR(AVG(items))

FROM

(SELECT

orderNumber, COUNT(orderNumber) AS items

MySQL Union

A JOIN combines result sets horizontally, a UNION appends result set vertically.

```
.....  
  
SELECT  
  
    firstName,  
  
    lastName  
  
FROM  
  
    employees  
  
UNION  
  
SELECT  
  
    contactFirstName,  
  
    contactLastName  
  
FROM  
  
    customers;
```

Inner Join

```
select customername,ordernumber,orderdate,status from customers c inner join orders o
on c.customernumber = o.customernumber
where status = 'In Process';
```

```
SELECT
  orderNumber,
  orderDate,
  customerName,
  orderLineNumber,
  productName,
  quantityOrdered,
  priceEach
FROM
  a   orders
  INNER JOIN orderdetails
    USING (orderNumber)
  INNER JOIN products
    USING (productCode)
  INNER JOIN customers
    USING (customerNumber)
WHERE orderNumber = 10425;
.....
```

```
SELECT
  o.orderNumber,
  orderDate,
  customerName,
  orderLineNumber,
  productName,
  quantityOrdered,
  priceEach
FROM
  orders o, customers c, orderdetails ord, products p
where
      o.customernumber = c.customernumber and
      o.ordernumber = ord.ordernumber and
      ord.productcode = p.productcode and
      o.orderNumber = 10425;
```

Left Join

```
SELECT

customerName,

orderNumber,

status

FROM

customers c

LEFT JOIN orders o

ON c.customerNumber = o.customerNumber

where country='USA';
```

```
.....

SELECT

customerName,

orderNumber,

status

FROM

customers

LEFT JOIN orders using(customerNumber)

where country='USA';
```

Right Join

SELECT

 employeeNumber,

 customerNumber

FROM

 customers

RIGHT JOIN employees

 ON salesRepEmployeeNumber = employeeNumber

ORDER BY

 employeeNumber;

Self Join

SELECT

e.firstName as 'EmpName',e.lastName,

m.firstName as 'Manager'

FROM

employees e

INNER JOIN employees m ON

m.employeeNumber = e.reportsTo;

Cross Join(all rows from both)

```
SELECT *
```

```
FROM employee
```

```
CROSS JOIN students;
```

```
+-----+-----+-----+-----+-----+-----+
| id | first_name | last_name | username | password | address | contact | id | name | city |
+-----+-----+-----+-----+-----+-----+
| 2 | Nair      | N        | user2    | pass     | address1 | 98989   | 1 | John | Hyd |
| 1 | Praveen   | N        | user1    | pass     | address  | 9999999999 | 1 | John | Hyd |
| 2 | Nair      | N        | user2    | pass     | address1 | 98989   | 2 | Ron  | Hyd |
| 1 | Praveen   | N        | user1    | pass     | address  | 9999999999 | 2 | Ron  | Hyd |
+-----+-----+-----+-----+-----+-----+
```

Modifying Data

```
set SQL_SAFE_UPDATES=0
```

```
update employees set email='martin@gmail.com' where employeenumber=1702;
```

```
insert into
```

```
employees(employeenumber,lastname,firstname,extension,email,officecode,reportsto,jobtitle)  
values(1111,'lastname','firstname','extn','email@email.com',4,1102,'programmer');
```

```
delete from employees where employeenumber=1111;
```

Managing Database

```
create database mydb;
```

```
drop database mydb;
```

Managing Table

```
CREATE TABLE users (
```

```
    userid int,
```

```
    username varchar(255),
```

```
    City varchar(255)
```

```
);
```

```
drop table users
```

Data Types

DATE - format YYYY-MM-DD

DATETIME - format: YYYY-MM-DD HH:MI:SS

TIMESTAMP - format: YYYY-MM-DD HH:MI:SS

YEAR - format YYYY or YY

INT

DECIMAL

BOOLEAN

VARCHAR

DATETIME YYYY-MM-DD HH:MM:SS

JSON
create table jsnn (js json);use emp;insert into jsnn(js) values(["name":"john"]);select * from jsnn

ENUM

.....

CREATE TABLE users(

userid int auto_increment primary key,

username varchar(255),

primary key(userid),

entry datetime,

Auto Increment

```
CREATE TABLE emp (  
    ID int NOT NULL AUTO_INCREMENT,  
    empname varchar(255) NOT NULL  
);  
  
-- start from 100  
ALTER TABLE Persons AUTO_INCREMENT=100;
```

Alter Table

ALTER TABLE users

ADD country varchar(255);

ALTER TABLE users

DROP COLUMN country;

ALTER TABLE users

MODIFY COLUMN country int;

Not Null Constraint

```
CREATE TABLE emp (  
    ID int NOT NULL,  
    empname varchar(255) NOT NULL,  
    salary int  
);  
  
ALTER TABLE emp  
MODIFY salary int NOT NULL;
```

UNIQUE Constraint

```
CREATE TABLE emp (
```

```
    ID int NOT NULL,
```

```
    empname varchar(255) NOT NULL,
```

```
    salary int,
```

```
    UNIQUE (ID)
```

```
);
```

```
CREATE TABLE emp (
```

```
    ID int NOT NULL,
```

```
    empname varchar(255) NOT NULL,
```

```
    salary int,
```

```
    CONSTRAINT UC_emp UNIQUE (ID,empname)
```

```
);
```

```
ALTER TABLE emp
```

```
ADD UNIQUE (ID);
```

```
ALTER TABLE emp
```

Primary Key constraint

```
CREATE TABLE emp (
```

```
  ID int NOT NULL,
```

```
  empname varchar(255) NOT NULL,
```

```
  salary int,
```

```
  PRIMARY KEY (ID)
```

```
);
```

```
CREATE TABLE emp (
```

```
  ID int NOT NULL,
```

```
  empname varchar(255) NOT NULL,
```

```
  salary int,
```

```
  CONSTRAINT PK_emp PRIMARY KEY (ID,empname)
```

```
);
```

```
ALTER TABLE emp
```

```
ADD PRIMARY KEY (ID);
```

```
ALTER TABLE emp
```

Foreign Key constraint

```
CREATE TABLE department (  
  
    depid int NOT NULL,  
  
    depname int NOT NULL,  
  
    empid int,  
  
    PRIMARY KEY (depid),  
  
    FOREIGN KEY (empid) REFERENCES emp(empid)  
  
);
```

```
CREATE TABLE department (  
  
    depid int NOT NULL,  
  
    depname int NOT NULL,  
  
    empid int,  
  
    PRIMARY KEY (depid),  
  
    CONSTRAINT FK_empdep FOREIGN KEY (empid)  
  
    REFERENCES emp(empid)  
  
);
```

```
ALTER TABLE department  
  
ADD FOREIGN KEY (empid) REFERENCES emp(empid);
```

Check constraint

```
CREATE TABLE emp (
```

```
  ID int NOT NULL,
```

```
  empname varchar(255) NOT NULL,
```

```
  Age int,
```

```
  CHECK (Age>=25)
```

```
);
```

```
CREATE TABLE emp (
```

```
  ID int NOT NULL,
```

```
  empname varchar(255) NOT NULL,
```

```
  Age int,
```

```
  country varchar(255),
```

```
  CONSTRAINT CHK_emp CHECK (Age>=25 AND country='India')
```

```
);
```

```
ALTER TABLE emp
```

```
ADD CHECK (Age>=25);
```

Default constraint

```
CREATE TABLE emp (  
  
    ID int NOT NULL,  
  
    empname varchar(255) NOT NULL,  
  
    Age int,  
  
    department varchar(255) default 'IT'  
  
);
```

```
CREATE TABLE emp (  
  
    ID int NOT NULL,  
  
    empname varchar(255) NOT NULL,  
  
    Age int,  
  
    department varchar(255) default 'IT',  
  
    doj date default current_date()  
  
);
```

```
ALTER TABLE emp  
  
ALTER department SET DEFAULT 'IT';
```

Create Index

```
CREATE INDEX idx_empname
```

```
ON emp (empname);
```

```
CREATE INDEX idx_empcountry
```

```
ON emp (empname,country);
```

```
ALTER TABLE emp
```

```
DROP INDEX idx_empname;
```

```
show indexes from employees;
```

Transactions

SET autocommit = 0; 1

SET autocommit = OFF on

delete query

rollback;

start transaction;

insert

delete

commit;

Transactions Example

START TRANSACTION; -- To disable autocommit mode implicitly for a single series of statements, use the START TRANSACTION statement:

```
SELECT @A:=SUM(salary) FROM table1 WHERE type=1;
```

```
UPDATE table2 SET summary=@A WHERE type=1;
```

```
COMMIT;
```

Stored Procedure

```
use classicmodels;  
  
DELIMITER &&  
  
CREATE PROCEDURE get_high_credit_cust ()  
  
BEGIN  
    SELECT * FROM customers WHERE creditlimit > 100000;  
  
END &&  
  
DELIMITER ;  
  
show procedure status where db='classicmodels';  
  
call get_high_credit_cust();
```

Stored Procedure (In Param)

```
use classicmodels;

set SQL_SAFE_UPDATES=0

DROP PROCEDURE get_high_credit_cust;

DELIMITER &&

CREATE PROCEDURE get_high_credit_cust (IN scr int)

BEGIN

    SELECT * FROM customers WHERE creditlimit > scr;

END &&

DELIMITER ;

call get_high_credit_cust(120000);
```

Stored Procedure (Out Param)

```
use classicmodels;

DROP PROCEDURE get_tot_cust;

DELIMITER &&

CREATE PROCEDURE get_tot_cust (OUT totcust int)

BEGIN

    SELECT count(customernumber) into totcust FROM customers;

END &&

DELIMITER ;

call get_tot_cust(@totcust);

select @totcust
```

Stored Procedure Status

```
SHOW PROCEDURE STATUS WHERE db='classicmodels';
```

```
DROP PROCEDURE get_tot_cust;
```

Triggers (Creating)

```
CREATE TABLE cust_audit (  
  customerNumber int,  
  contactLastName varchar(255),  
  changedAt datetime  
);
```

```
-----  
CREATE TRIGGER before_cust_update  
  BEFORE UPDATE ON customers  
  FOR EACH ROW  
  INSERT INTO cust_audit  
  SET action = 'update',  
      customerNumber = OLD.customerNumber,  
      contactlastname = OLD.contactLastName,  
      changedat = NOW()
```

```
Show triggers;
```

```
Drop trigger before_cust_update;
```

```
update customers set contactLastName='John' where customerName like '%Kelly%';
```

```
Select * from cust_audit;
```

Triggers (automatically calls)

Create cust_audit table first

set SQL_SAFE_UPDATES=0

Drop trigger name;

update customers set contactlastname='Kngg' where customernumber=175;

Creating View

```
create view vcust as select * from customers where customernumber=103  
select * from vcust;
```

```
SHOW FULL TABLES
```

```
WHERE table_type = 'VIEW';
```

```
drop view vcust;
```

Thank You

- PRAVEEN NAIR